

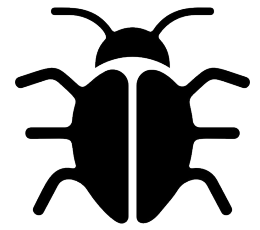
Laboratorio I

a.a. 2025/2026

Eccezioni

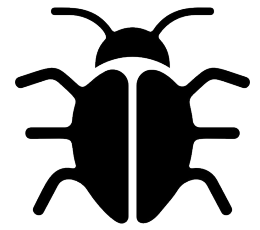
Contenuti

- Bug & Errori
- Gestire errori
 - Sistema di eccezioni
 - Catturare un'eccezione
 - Lanciare un'eccezione
 - Distinguere tra eccezioni diverse



Bug e errori

- Durante lo sviluppo di un programma possono essere introdotti dei difetti - bug
 - Errori di sintassi - di solito riconosciuti subito dal sistema
 - Errori di battitura - typo - un nome di una variabile/funzione scritto in modo sbagliato possono generare un errore ma non sempre in JavaScript
 - Errori di logica - l'algoritmo proposto non funziona in alcuni casi - di solito silenziosi fino a quando non si incontra il caso limite



Bug e errori

- Altri errori esterni al nostro programma
 - Errori di input/output
 - l'utente inserisce dei dati mal formattati
 - Un file è stato cancellato per sbaglio
 - Il disco rigido è fallito
 - La rete non funziona per un cavo rotto, un router 'deceduto'
 - ecc

Esempi di errore (su JDoodle)

Scriviamo una funzione `calc(a)` che, dato come argomento un array `a`, in cui il primo elemento è un operatore aritmetico (+, -, *, /) rappresentato come carattere, restituisca il risultato ottenuto applicando l'operatore a tutti i rimanenti elementi dell'array.

Esempi

`calc(["+",4,5,2]) → 11`

`calc(["*",4,5,2]) → 40`

`calc(["/",64,2,2,4]) → 4`

*Splendida occasione per un mitico
assegnamento destrutturante*

`[op,...x] = a`

e per (ri)vedere la reduce

Esempi di errore - cont. (su JDoodle)

```
function calc(a) {  
  [op,...x] = a  
  switch (op) {  
    case '+':  
      return x.reduce((y,z)=>(y+z),0)  
    case '-':  
      [x1,...x] = x  
      return x.reduce((y,z)=>(y-z),x1)  
    case '*':  
      return x.reduce((y,z)=>(y*z),1)  
    case '/':  
      [x1,...x] = x  
      return xx.reduce((y,z)=>(y/z),x1)  
  }  
}
```

```
> calc(['+',1,2,3,4])  
10  
> calc(['+', 'ciaone', 2, 3, 4])  
'0ciaone234'  
> calc(['-', 'ciaone', 2, 3, 4])  
NaN
```

OK

Oops

...con i tipi ce ne accorgeremmo
prima, un po' come in

```
> calc(['/',5,6,7])  
ReferenceError: xx is not defined  
    at calc (/home/runner/prove-1/index.js:13:7)
```

Gestione degli errori

- Debugging - trovare i bug - processo che può essere molto difficile
 - Scrivere codice ben organizzato ci aiuta a trovare i bug!
 - Modalità **strict**: `'use strict'`
- Alcuni tipi di errori non possono essere evitati
 - JavaScript prova sempre a trovare una soluzione però a volte il sistema dà un risultato sbagliato o un messaggio di errore.
 - Non è una buona idea esibire i messaggi di errore all'utente in un sistema software
 - Bisogna gestire gli errori, dare feedback all'utente e poi continuare l'esecuzione dal punto giusto

Sistema di gestione delle **eccezioni**

Sistema di gestione delle eccezioni

- Eccezione
 - Meccanismo di interruzione del programma in caso di errore
 - Viene 'lanciata' quando l'errore compare
 - Automatiche
 - Possibilità di lanciare anche 'manualmente' a bisogno
 - Può essere 'catturata' nel posto più adatto del programma
 - Anche semplicemente per evitare messaggi di errore 'brutti'

Esempi di eccezione (su JDoodle)

Scriviamo una funzione `calcAll(e)` che, dato come argomento un array `e` di espressioni, invoca `calc` su ciascuna espressione

```
function calcAll(e) {  
  return e.map(calc)  
}
```

Cosa succede se facciamo `calcAll(['+',3,4,5],['- ',5,4],undefined)` ?

Esempi di eccezione - cont (su JDoodle)

```
TypeError: undefined is not iterable (cannot read prop  
Symbol(Symbol.iterator))  
    at calc (/home/runner/prove-1/index.js:2:13)  
    at Array.map (<anonymous>)  
    at calcAll (/home/runner/prove-1/index.js:18:12)  
    at Object.<anonymous> (/home/runner/prove-1/index.js:22  
:6)  
    at Module._compile (internal/modules/cjs/loader.js:999:  
30)
```

Questa è un'eccezione..

Catturare un'eccezione

- Come? Usando il comando **try-catch**

```
try{  
  a=calcAll(expressions)  
  console.log(a)  
} catch (e){  
  console.log("Sorry not working today!")  
}
```

Blocco può contenere
anche chiamate a funzioni
che lanciano eccezioni

```
try {  
    //comandi  
  
} catch (e){  
    //comandi gestione errore  
}
```

Cosa succede?

- Il codice del blocco **try** viene eseguito, fino al primo comando che genera un errore, incluso nelle chiamate annidate di funzioni. I comandi oltre l'errore non vengono eseguiti e viene restituito il controllo alla funzione corrente.
- Se compare un errore, si esegue il blocco **catch**. La variabile **e** contiene informazioni sull'errore.
- Se non c'è nessun errore, il blocco **catch** viene saltato e si continua l'esecuzione del programma.

Catturare un'eccezione

- L'eccezione viene memorizzata in una variabile di tipo Error (o un suo sottotipo)
- Proprietà:
 - message
 - stack
 - name
 - ...et al.

```
try{  
  a=calcAll(expressions)  
  console.log(a)  
} catch (e){  
  console.log("Sorry not working today")  
  console.log(e.message)  
}
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error

Lanciare un'eccezione

- Tante volte può essere utile poter lanciare un'eccezione noi stessi
 - Per poter segnalare un problema specifico al nostro programma
 - L'esecuzione si ferma e si salta al blocco catch più vicino

```
default:  
    throw new Error("Operator is not correct,  
    should be one of +-* / ")
```

Lanciare eccezioni (su JDoodle)

Proviamo a gestire due bug del nostro esempio lanciando eccezioni

```
> calc(['+', 'a', 2, 3])  
'0a23'  
> calc(['%', 1, 2, 3])  
undefined
```

Lanciare eccezioni (su JDoodle)

```
function calc(a) {  
  [op,...x] = a  
  for (i of x) {  
    if(typeof i !== "number")  
      throw new Error("Gli operandi devono essere numeri")  
  }  
  switch (op) {  
    case '+':  
      return x.reduce((y,z)=>(y+z),0)  
    case '-':  
      [x1,...x] = x  
      return x.reduce((y,z)=>(y-z),x1)  
    case '*':  
      return x.reduce((y,z)=>(y*z),1)  
    case '/':  
      [x1,...x] = x  
      return xx.reduce((y,z)=>(y/z),x1)  
    default:  
      throw new Error("Operatore sconosciuto")  
  }  
}
```

Se almeno un elemento di x non è un numero, lanciamo eccezione

Default = operatore sconosciuto

Distinguere tra eccezioni

- Il tipo di un'eccezione ci aiuta a distinguere tra vari tipi di errori e gestirli in modo diverso

```
try{
    a=calcAll(expressions)
    console.log(a)
} catch (e){
    if (e instanceof TypeError)
        console.log("Wrong input!")
    else if (e instanceof ReferenceError)
        console.log("You have an unreferenced variable!!")
}
```


Creare un tipo di eccezione

```
default:  
    throw new Error("Operator is not correct,  
    should be one of +-*/ ")
```

- `new Error()` utile ma non permette di riconoscere errori di natura diversa e nasconde gli altri errori
- E' utile creare i nostri tipi di oggetti che implementano **Error**
- Gli altri errori possono essere rilanciati

Creare un tipo di eccezione

```
class OperatorError extends Error{}
```

- Possiamo definire un'intera gerarchia di errori
 - Il design del programma include il design del sistema di gestione delle eccezioni

```
try{  
  a=calcAll(expressions)  
  console.log(a)  
} catch (e){  
  if (e instanceof OperatorError)  
    console.log("Wrong operator input!")  
  else throw e  
}
```

Definire eccezioni (su JDoodle)

Proviamo a gestire due bug del nostro esempio lanciando eccezioni opportune

```
➤ calc(['+', 'a', 2, 3])  
'0a23'
```

-> OperandError

```
➤ calc(['%', 1, 2, 3])  
undefined
```

-> OperatorError

Definire eccezioni - cont. (su JDoodle)

Definiamo una gerarchia di eccezioni

```
class ExpressionError extends Error {}  
class OperandError extends ExpressionError {}  
class OperatorError extends ExpressionError {}
```

Definire eccezioni - cont. (su JDoodle)

Lanciamo le eccezioni definite

.....

```
for (i of x) {  
  if (typeof i !== "number")  
    throw new OperandError("Gli operandi devono essere numeri")  
}
```

.....

```
default:  
  throw new OperatorError("Operatore sconosciuto")
```

Definire eccezioni - cont. (su JDoodle)

```
function calcAll(e) {  
  res = null  
  try {  
    res = e.map(calc)  
  } catch(e) {  
    if (e instanceof OperandError) {  
      console.log("Non funziono per gli operandi")  
      console.log(e.message)  
    } else if (e instanceof OperatorError) {  
      console.log("Non funziono per l'operatore")  
      console.log(e.message)  
    } else {  
      console.log("Non funziono, punto")  
    }  
  }  
  return res  
}
```

finally

- Tante volte uscire dal programma/funzione subito dopo un errore non è indicato
 - Bisogna lasciare i dati su cui si è lavorato in uno stato valido, chiudere file, etc.
 - Il blocco **finally** ci permettere di includere comandi da eseguire sempre, sia nel caso di errore che non.

Gestione delle eccezioni - sintassi completa

```
try{
    a=calcAll(expressions)
    console.log(a)
} catch (e){
    if (e instanceof OperatorError)
        console.log("Wrong operator input!")
    else if (e instanceof OperandError)
        console.log("Wrong operand input!")
    else throw e
} finally {
    console.log("Cannot leave without saying
    bye!")
}
```

```
try {

    //comandi

} catch (e){

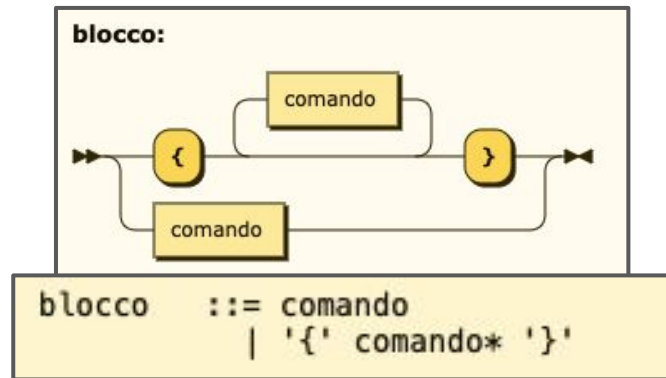
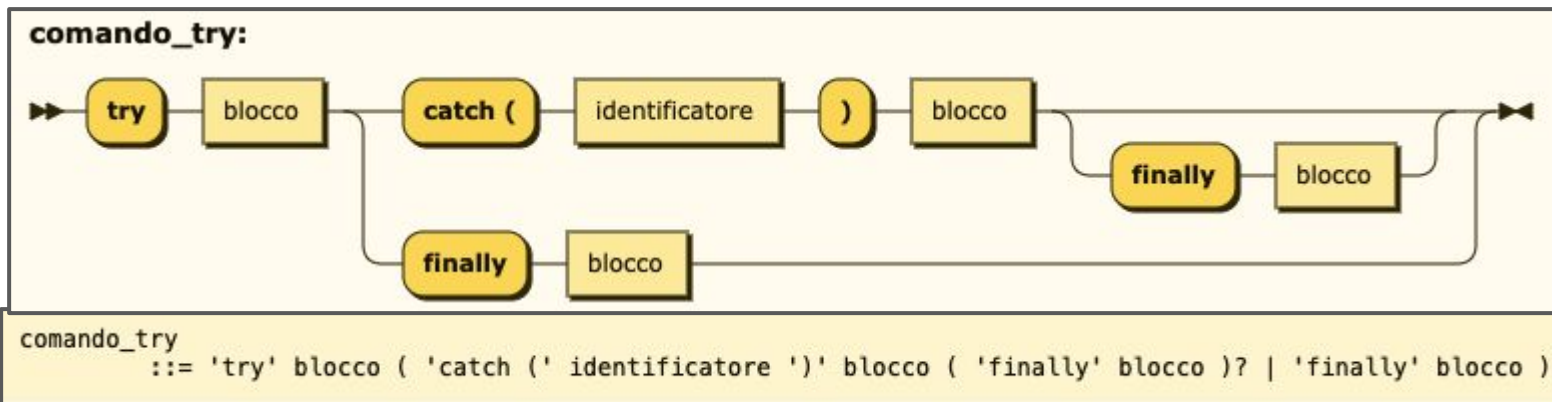
    //comandi gestione errore

} finally {

    //comandi finali

}
```


Gestione delle eccezioni - sintassi completa



```
try {  
    //comandi  
} catch (e){  
    //comandi gestione  
errore  
} finally {  
    //comandi finali  
}
```

- I blocchi **finally** e **catch** possono essere omessi, ma almeno uno ci deve essere
- I comandi **try-catch-finally** possono essere annidati

Q & A